Práctica 4. Node.js

Desarrollo de Sistemas Distribuidos. Grupo DSD-3.

Grado en Ingeniería Informática

Curso 2025-2026.

ETSIIT. Universidad de Granada.





UNIVERSIDAD
DE GRANADA

Autor:
Ángel Rodríguez Faya

Profesor:
José Ángel Segura Muros

Fecha de entrega:
22 de mayo de 2026

Índice

1. Introducción. **2**

2. Parte 1.- Implementación de ejemplos. **3**

2.1. Ejemplo 1.- Hola Mundo. 3
2.2. Ejemplo 2.- Calculadora REST. 5
2.3. Ejemplo 3.- Calculadora WEB. 6
2.4. Ejemplo 4.- Socket.io. 8
2.5. Ejemplo 5.- MongoDB. 10

1. Introducción.

Este documento constituye la memoria de la Práctica 4 de la asignatura *Desarrollo de Sistemas Distribuidos*. El objetivo central de este proyecto radica en el diseño, implementación y evaluación experimental de un sistema domótico distribuido, cooperativo y en tiempo real, haciendo uso de tres pilares tecnológicos esenciales:

- **Node.js:** Es una plataforma de ejecución de JavaScript del lado del servidor basada en el motor V8 de Google. Destaca por su modelo de operaciones de I/O no bloqueantes y su arquitectura orientada a eventos (*event-driven*), controlada por un único hilo de ejecución centralizado (*Event Loop*). Esto permite mitigar la sobrecarga asociada a la creación de hilos tradicionales por cada conexión entrante.
- **Socket.io:** Es una biblioteca especializada en habilitar una capa de comunicación bidireccional, síncrona y de alta velocidad entre clientes y servidores sobre el protocolo WebSocket. Su modelo basado en eventos encaja de forma nativa con el patrón de diseño Publicación/Suscripción, idóneo para la propagación inmediata de estados concurrentes.
- **MongoDB:** Es un sistema de base de datos `NoSQL` orientado a documentos que almacena la información en esquemas flexibles de tipo `BSON/JSON`. Proporciona un mecanismo de persistencia ideal para el flujo heterogéneo de telemetría y logs de eventos generados de forma asíncrona en entornos distribuidos.

El propósito último de este proyecto/práctica es el despliegue de una infraestructura inteligente donde múltiples procesos independientes — como nodos sensores, nodos actuadores, un agente lógico de control supervisado y paneles web de usuario (o *dashboards*)— cooperen de manera asíncrona y transparente para garantizar la habitabilidad y eficiencia del entorno que está siendo monitorizado.

2. Parte 1.- Implementación de ejemplos.

Antes de abordar el diseño de la arquitectura completa del sistema domótico, es fundamental que asimilemos los fundamentos de red y los patrones de concurrencia asíncrona que manejan el entorno de Node.js. Por este motivo, la primera fase o parte de la práctica se centra en la implementación, despliegue y análisis de una serie de servicios y ejemplos base propuestos en el guion.

A continuación, vamos a ir viendo cada uno de los ejemplos. Para ello, analizaremos el código fuente de cada uno de los ejemplos, verificar su correcto funcionamiento mediante capturas de pantalla, y la correspondiente justificación técnica de los fenómenos observados.

2.1. Ejemplo 1.- Hola Mundo.

Para poner en marcha este primer ejemplo, abrimos la terminal en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_1_Hola_Mundo`) y ejecutamos el comando:

```
$ node helloworld.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla que verifican el correcto despliegue del servidor y su respuesta:



Figura 1: Visualización en el navegador de la respuesta enviada por el servidor.

```
angelrf@angel-ubuntu:~/Practica_4_Node_js/codigo/Parte_1_Implementacion_de_Ejemp
los/Ejemplo_1_Hola_Mundo$ node helloworld.js
Servicio HTTP iniciado
{
  host: 'localhost:8080',
  'user-agent': 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/2010010
1 Firefox/150.0',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
  'accept-language': 'es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7',
  'accept-encoding': 'gzip, deflate, br, zstd',
  connection: 'keep-alive',
  cookie: 'phpMyAdmin=6fc530396eb76ccbaff352cbac7b55b4; PHPSESSID=da9e2736ada057
f6910b6463a4dcf300',
  'upgrade-insecure-requests': '1',
  'sec-fetch-dest': 'document',
  'sec-fetch-mode': 'navigate',
  'sec-fetch-site': 'none',
  'sec-fetch-user': '?1',
  priority: 'u=0, i'
}
```

Figura 2: Salida por terminal del servidor mostrando el inicio y las cabeceras recibidas.

Vemos que el script utiliza el módulo nativo `http` de Node.js para levantar un servidor web básico que se queda escuchando peticiones en el puerto `8080`. Cada vez que un cliente (como el navegador) accede, el servidor imprime en la consola las cabeceras de la petición y devuelve un mensaje de texto plano con el contenido “Hola mundo”.

Lo más relevante a destacar de este ejemplo es:

- **La petición no bloqueante (Asíncrona):** El mensaje `'Servicio HTTP iniciado'` se muestra en la terminal de forma inmediata al lanzar el script. Esto demuestra que el método `.listen(8080)` es asíncrono y no detiene la ejecución del programa a esperar conexiones.
- **Uso de Callbacks:** La función interna de `createServer` actúa como un *callback*. No se ejecuta al arrancar el programa, sino que el *Event Loop* (bucle de eventos) la invoca únicamente cuando entra una petición HTTP, gestionando así la concurrencia de forma eficiente en un solo hilo o hebar.

2.2. Ejemplo 2.- Calculadora REST.

Para este segundo ejemplo, lo podemos poner en marcha abriendo en la terminal el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_2_Calculadora_REST`) y ejecutamos el comando:

```
$ node calculadora.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla:

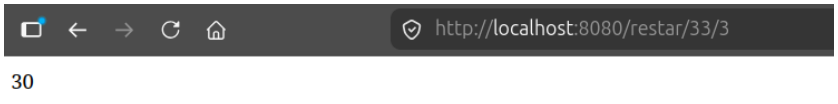


Figura 3: Visualización en el navegador del resultado de restar 33-3

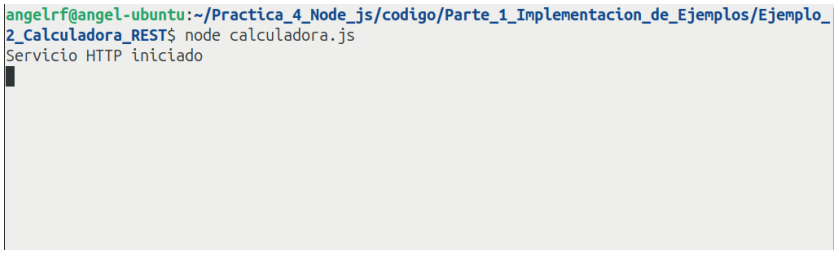


Figura 4: Salida por terminal del servidor mostrando el inicio.

Viendo el código fuente y la figura 3 podemos sacar varias conclusiones/apuntes sobre Node.js:

- El servidor tiene que **procesar la URL** como una cadena de texto, para descomponer la ruta de la petición en un array de parámetros.
- Como todo lo que viaja por HTTP es texto, necesitamos **deserializar los datos** mediante `parseFloat()` para operar con ellos numéricamente y, posteriormente, convertirlos de nuevo a texto (`result.toString()`) para poder transmitirlos en la respuesta HTTP.
- Se realiza un **control de errores**. Se comprueba que el array de parámetros extraído de la URL tenga >= 3 parámetros para poder realizar la operación. En caso de que tenga menos, el sistema no se cae (no hace *crash*), si no que responde con un mensaje de error controlado manteniendo así la **disponibilidad del servicio**

Este ejemplo es una calculadora **REST**. Esto quiere decir que el servidor actúa como una **API**. Solo espera recibir comandos en la URL (ej. /sumar/5/3) y devuelve datos puros (el resultado en texto plano). No tiene diseño, ni botones, ni colores. Está pensado para comunicación máquina a máquina (por ejemplo, para que otro programa o un comando de terminal interactúe con él), tal y cómo hemos visto en las capturas de pantalla.

2.3. Ejemplo 3.- Calculadora WEB.

En este ejemplo, vemos otra calculadora pero a diferencia de la anterior, esta es una calculadora WEB. Es decir, el servidor actúa como un **Servidor Web tradicional y además como API**. Está orientado a humanos y no a máquinas como el ejemplo anterior. Cuando entras a la ruta base (/), el servidor lee del disco duro y nos devuelve una página web completa (`calc.html`). Esa página web tiene una interfaz visual (un formulario,

un desplegable) y contiene código JavaScript que, de forma invisible (usando peticiones `fetch`), hace las llamadas REST al servidor para obtener el resultado sin tener que recargar la página.

Para poder ejecutar este ejemplo, abrimos la terminal y nos situamos en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_3_Calculadora_WEB`) y ejecutamos el comando:

```
$ node calculadora-web.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla:

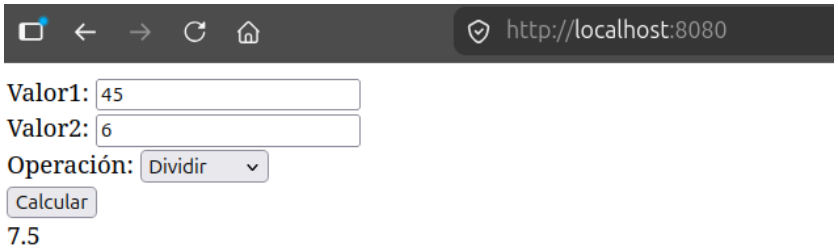


Figura 5: Visualización en el navegador de la calculadora comportándose como una página web normal. Divide 45/6.

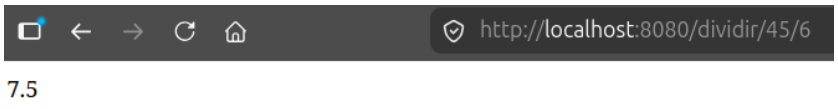


Figura 6: Visualización en el navegador de la calculadora comportándose como una API. Divide 45/6.



Figura 7: Salida por terminal del servidor mostrando el inicio.

Este ejemplo da un paso más hacia una arquitectura web real al separar el *Frontend* (interfaz) del *Backend* (lógica). El servidor ahora realiza dos funciones distintas dependiendo de la ruta (*routing*) solicitada:

- En la figura 5 podemos ver el **servicio de archivos estáticos (I/O Asíncrono)**. Si el cliente accede a la raíz (`/`), el servidor utiliza los módulos nativos `path` y `fs` (File System) para buscar el archivo `calc.html` en el disco duro. Emplea la función asíncrona `readFile` junto con un *callback* `(err, data)` para leer el archivo sin bloquear el hilo principal, y lo devuelve al cliente con la cabecera `text/html`. Comportándose como una página web real.

- En cambio, en la figura 6, vemos en acción el **consumo de API mediante AJAX (`fetch`).** En el lado del cliente (el navegador), el archivo HTML contiene un script que intercepta el envío del formulario (`submit`). En lugar de recargar la página, utiliza la API `fetch` de JavaScript (basada en Promesas con `.then()`) para construir la URL dinámica (ej. `/sumar/5/3`), solicitar el resultado al servidor en segundo plano y actualizar únicamente la etiqueta `<span>` del resultado en el DOM. Este comportamiento es parecido al de la calculadora del ejemplo anterior.

2.4. Ejemplo 4.- Socket.io.

Ahora ya estamos ante una página web **en tiempo real y bidireccional**. En este ejemplo voy a comentar primero las anotaciones o cueriosidades y luego vemos los ejemplos.

- Ya nos olvidamos de las peticiones, pues ahora trabajaremos con **eventos**. Esto ocurre porque, a diferencia de HTTP que obliga a que el navegador siempre pregunte primero para que el servidor responda, Socket.io (basado en WebSockets) mantiene un "tubo"de comunicación siempre abierto. Ahora, el servidor puede enviar datos al navegador por iniciativa propia sin que nadie se lo pida.

- Estamos ante el **Patrón de Suscripción/Emisión** (`emit` y `on`), que se diferencia con el **Patrón *Observer*** en que los publicadores (emisores) y los suscriptores (receptores) no se comunican directamente entre sí. Utilizan un intermediario (*Message Broker* o Bus de Eventos). Los publicadores envían mensajes a un çanal.º "temaz los suscriptores escuchan ese canal. Es por eso que la comunicación ya no se basa en URLs (rutas), sino en .ev"entosçon nombres inventados por el programador.
  - Usamos `.emit('nombre-evento', datos)` para disparar un mensaje por el "tubo".
  - Usamos `.on('nombre-evento', callback)` para quedarnos escuchando.
  - Por ejemplo, el navegador emite `output-evt` con el texto "¡Hola Servicio!", y el servidor lo escucha para contestar.
- Se realiza una **Difusión o *Broadcast*** (`io.sockets.emit`): El servidor de Node.js guarda un array llamado `allClients` con las direcciones IP y puertos de todos los conectados. Cuando alguien nuevo entra (o cuando alguien cierra la pestaña y se dispara el evento `disconnect`), el servidor usa `io.sockets.emit('all-connections', allClients)`, para mantenerla lista actualizada a todos los clientes que están conectados a la vez.
- El **DOM es Dinámico** (Frontend): En el archivo HTML, encontramos código JavaScript que está "suscrito.al evento `all-connections`. Cada vez que el servidor lanza esa nueva lista, el cliente borra la lista antigua de la pantalla y dibuja la nueva al instante, sin tener que darle al botón de actualizar del navegador.

Para poder ejecutar este ejemplo, abrimos la terminal y nos situamos en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_4_Socketio`) y ejecutamos el comando:

```
$ node connections.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla:

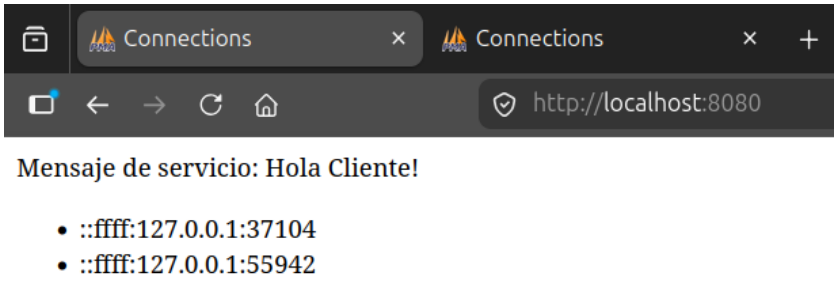


Figura 8: Visualización en el navegador de la dirección Ip y el puerto de los dos clientes conectados a nuestro servidor.

```
angelrf@angel-ubuntu:~/Practica_4_Node_js/codigo/Parte_1_Implementacion_de_Ejemplos/Ejemplo_
4_Socketio$ node connections.js
Servicio Socket.io iniciado
Nueva conexión de ::ffff:127.0.0.1:37104
Nueva conexión de ::ffff:127.0.0.1:55942
```

Figura 9: Salida por terminal del servidor mostrando el inicio.

Como podemos observar en las figuras, el cliente web muestra los clientes conectados. La dirección IP es la misma puesto que he abierto dos pestañas en mi misma máquina, eso sí, el puerto obviamente si que cambia

2.5. Ejemplo 5.- MongoDB.

En este ejemplo vamos un paso más allá: incorporar una base de datos real. Utilizaremos `MongoDB`, que es un sistema de gestión de bases de datos de tipo NoSQL . Este SGBD utiliza colecciones de entradas en formato JSON, en lugar de tablas que encontramos en SQL. Cada entrada puede tener un conjunto de claves y valores arbitrarios. Para obtener información, realizamos consultas similares a las utilizadas en SQL.

En este caso se utiliza para almacenar los clientes que se han conectado en nuestro servidor. En el ejemplo 4, los clientes conectados se almacenaban en un array de JS. Tenía como inconveniente que los datos eran volátiles, pues cuando apagamos el servidor, el array obviamente se borraba. En este ejemplo, podemos ver en las capturas de pantalla que las direcciones de los clientes se almacenan en la BD, porque se almacenan junto a la fecha y hora en la que se conectaron, y vemos que hay dos fechas distintas: 24 de abril y 21 de mayo.

Para poder ejecutar este ejemplo, abrimos la terminal y nos situamos en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_5_MongoDB`) y ejecutamos los comandos:

- Para arrancar el servicio de MongoDB (previamente debe estar instalado en nuestro PC)

```
$ sudo systemctl start mongod
```

- Para lanzar el servidor:

```
$ node mongo-test.js
```

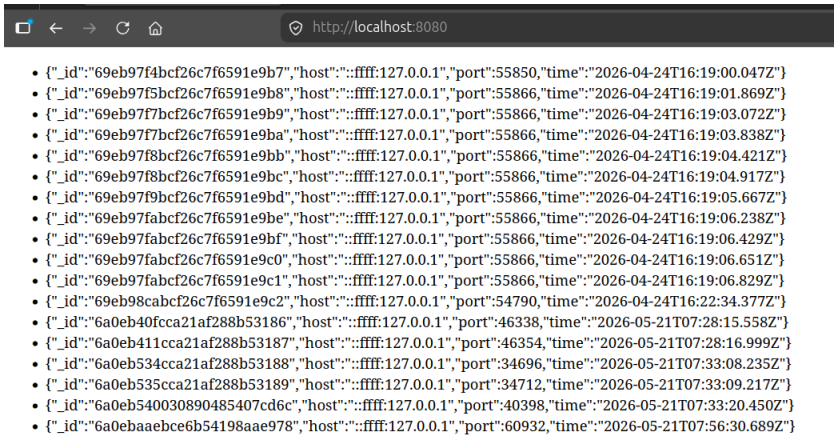


Figura 10: Visualización en el navegador de os clientes que se han conectado alguna vez al servidor.



Figura 11: Salida por terminal del servidor mostrando el inicio.

Observando las figuras 10 y 11, vemos que, a diferencia del Ejemplo 4 (donde solo imprimíamos texto plano formateado con la IP y el puerto), aquí vemos que los datos aparecen en pantalla como objetos JSON puros encadenados. El campo llamado `_id` con un código alfanumérico largo (`ObjectId`). Este elemento no lo ha programado el cliente; lo genera e inyecta MongoDB automáticamente para que actúe como clave primaria única del documento dentro de la colección.

Además, se utilizan Promesas o *Promises* para un *Flujo Asíncrono* , evitando bloquear el único hilo de ejecución de Node.js durante las operaciones de Entrada/Salida (I/O) con el disco. El driver de MongoDB utiliza las *Promises*. Toda la lógica de red se anida secuencialmente dentro de métodos `.then()`, asegurando que el servidor pueda seguir escuchando nuevos sockets mientras procesa consultas concurrentes.

En el Ejemplo 4, el servidor tomaba un rol activo enviando la lista de conexiones por iniciativa propia nada más establecerse el enlace. En el Ejemplo 5, el flujo lo conduce el cliente web de forma reactiva:

1. El servidor le dice al cliente cuál es su dirección (`my-address`).
2. El cliente añade la fecha actual y ordena activamente guardarla (`poner`).
3. Finalmente, el cliente solicita expresamente recuperar el histórico (`obtener`).

Por último, destacar que el Servidor no tiene estado (*Stateless*). Es lo que habíamos comentado al principio del ejemplo, dado que el servidor de Node.js ya no mantiene ninguna lista viva de sockets en su memoria volátil, se elimina por completo la necesidad de escuchar el evento `disconnect` en `mongo-test.js`. Al servidor le resulta indiferente si el usuario cierra el navegador, ya que el registro queda guardado de manera permanente e inmutable en MongoDB.